การยางแห่งประเทศไทย
Rubber Authority of Thailand

Ditruz
IT Services Provider

Frontend Development
ด้วย Vue + TypeScript

Frontend คืออะไร ?

เว็บไซต์ยุคใหม่ไม่ได้เป็นแค่ "หน้าเว็บ"

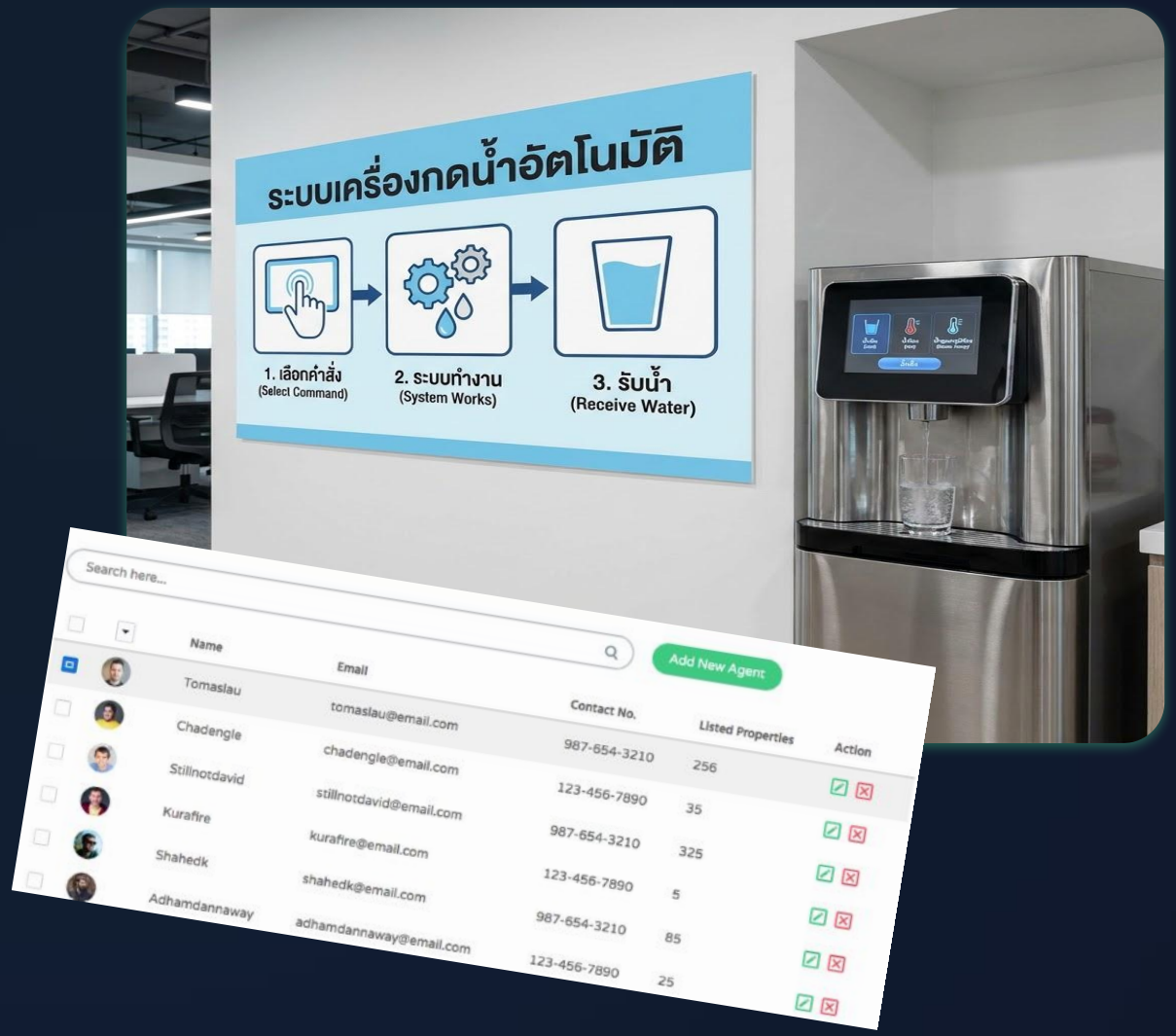
เปรียบเทียบ:

- **ยุคก่อน**: เหมือน "โปสเตอร์โฆษณา" มีไว้อ่านข้อมูลเฉยๆ เปลี่ยนแปลงอะไรไม่ได้มาก
- **ยุคนี้ (Web Application)**: เหมือน "เครื่องกดน้ำอัตโนมัติ" ระบบที่ผู้ใช้เลือกคำสั่ง แล้วระบบทำงาน

ตัวอย่างระบบที่ซับซ้อน (Frontend หนักๆ)

- ระบบ Dashboard สรุปยอดขาย
- ระบบจัดการสต๊อกสินค้า (Inventory)
- ระบบจุดขายหน้าเว็บ (POS)
- ระบบหลังบ้าน Admin Panel

ดังนั้น การสร้างระบบพวกนี้ จึงต้องใช้เครื่องมือพิเศษเข้ามาช่วยจัดการ!



Framework คืออะไร ?

ทำไมบริษัทใหญ่ๆ ถึงเลือกใช้ Vue Framework?

เรียนรู้ง่าย (Easy to Learn)

โครงสร้างโค้ดเข้าใจง่าย ไม่ซับซ้อนจนเกินไป ทำให้ทีมพัฒนาใหม่ๆ สามารถเข้ามาเรียนรู้และเริ่มงานได้เร็วกว่าเครื่องมืออื่นๆ

ทำงานเร็ว (High Performance)

ตัวระบบมีขนาดเล็กและเบา ทำให้โหลดหน้าเว็บได้เร็ว ผู้ใช้งานไม่รู้สึกหงุดหงิดเวลาคลิกหรือเปลี่ยนหน้าจอ

ขยายสเกลได้ (Scalable)

ด้วยโครงสร้างที่ชัดเจน ทำให้สามารถใช้สร้างตั้งแต่เว็บเล็กๆ ไปจนถึงระบบองค์กรขนาดใหญ่ที่มีความซับซ้อนสูงได้

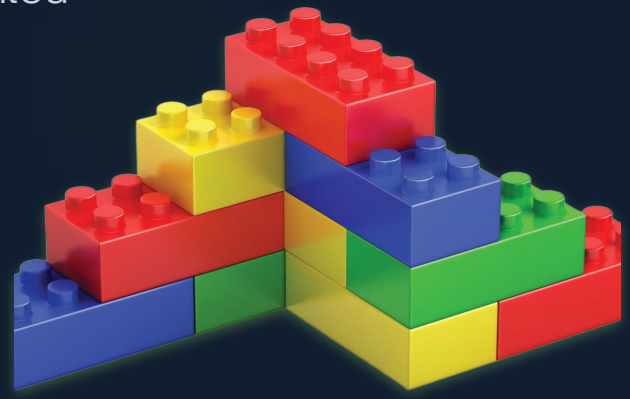
หัวใจของ Vue คือ Component-Based

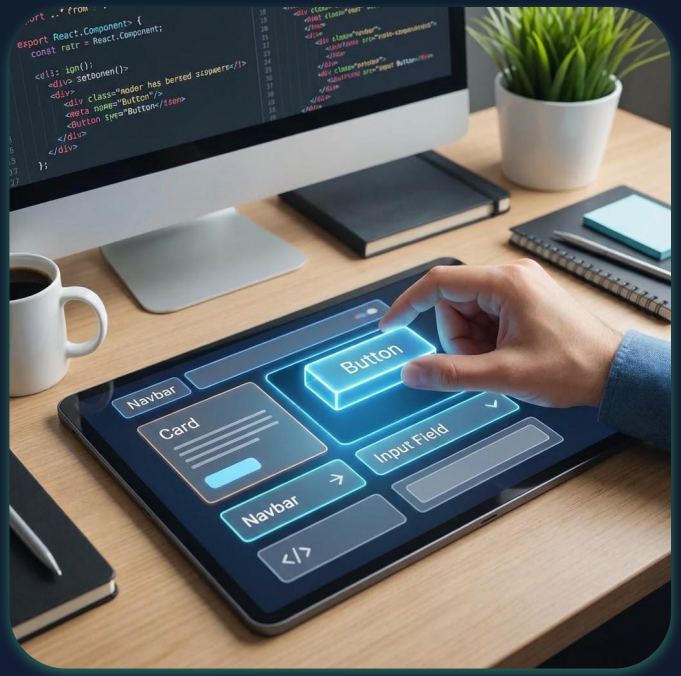
Component คืออะไร?

อธิบายง่ายๆ มันคือ "**ตัวต่อตัวต่อเลโก้ (Lego)**" ของหน้าเว็บไซต์

แทนที่เราจะเขียนโค้ดหน้าเว็บทั้งหน้ายาวๆ เราจะหั่นหน้าเว็บออกเป็นชิ้นส่วนเล็กๆ เช่น

- **ชิ้นส่วน A**: แถบเมนูด้านบน (Navbar)
- **ชิ้นส่วน B**: เมนูด้านข้าง (Sidebar)
- **ชิ้นส่วน C**: ตารางสินค้า (Product Table)





ข้อดีสุดปัง: เราสามารถเอาชิ้นส่วนเหล่านี้ไป "ประกอบซ้ำ (Reuse)" ในหน้าอื่นๆ ได้เลย ไม่ต้องเสียเวลาสร้างใหม่ แก้ไขจุดเดียว เปลี่ยนทั้งระบบ!

Vue (Vue Anatomy)

ใน 1 ไฟล์ของ Vue จะถูกแบ่งเป็น 3 ส่วนหลักที่แยกหน้าที่กันชัดเจน

<template>

เหมือน "ร่างกาย"

ส่วนนี้ใช้เขียน HTML เพื่อกำหนดโครงร่างหน้าตาของเว็บ ว่าจะมีปุ่มตรงไหน มีข้อความอะไรบ้าง

<script>

เหมือน "สมอง"

ส่วนนี้ใช้เขียน Logic (JavaScript/TypeScript) เพื่อสั่งการว่าคลิกปุ่มนี้แล้วจะเกิดอะไรขึ้น ดึงข้อมูลมาแสดงยังไง

<style>

เหมือน "เสื้อผ้า/เครื่องสำอาง"

ส่วนนี้ใช้เขียน CSS เพื่อตกแต่งความสวยงาม ใส่สีสัน จัดระยะห่าง ให้หน้าเว็บดูดีน่าใช้งาน

ทำไมเราต้องใช้ TypeScript?

JavaScript ธรรมดา

เปรียบเทียบ: เหมือนถนนที่ไม่มีเส้นแบ่งเลน ไม่มีตำรวจจราจร ขับรถยังไงก็ได้ (อิสระแต่เสี่ยงชนสูง)

ไม่มีการตรวจชนิดข้อมูล (Type) ปล่อยผ่านหมด

```
let price = 100;
    price = "hello" ;
```

ใช้ TypeScript

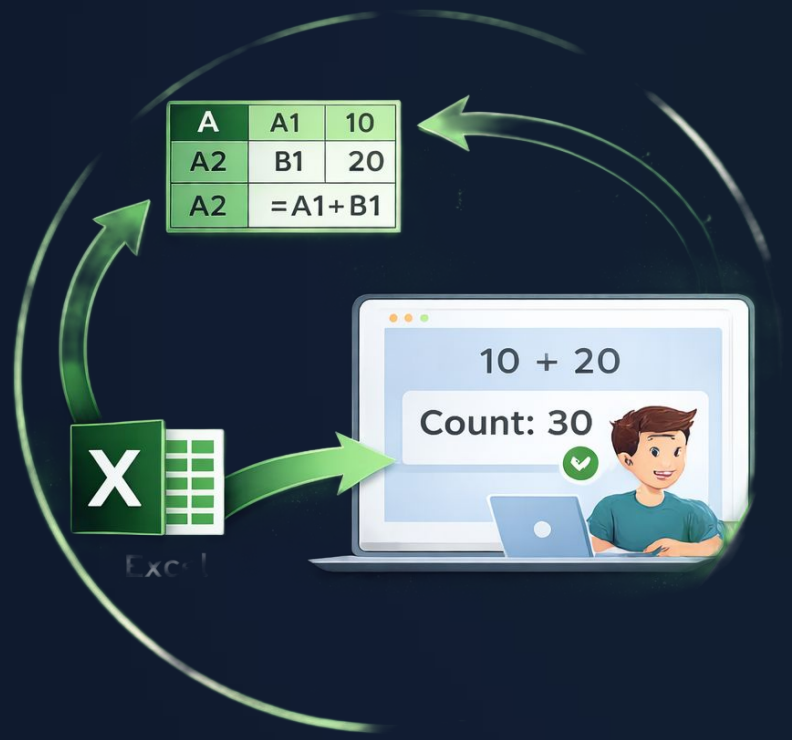
เปรียบเทียบ: เหมือนถนนที่มีการตีเส้นชัดเจน มีระบบ AI คอยเตือนถ้ารถจะออกนอกเลน (ปลอดภัยกว่า)

ตรวจจับข้อผิดพลาด (Bug) ให้ทันทีตั้งแต่ตอนพิมพ์โค้ด!

```
let price: number = 100;
           price = "hello";

// ❌ โปรแกรมเมอร์จะเห็นแถบแดง Error ทันที!
```

ความฉลาดของ Vue Reactive System



ระบบ Reactive (เปลี่ยนแปลงปุ๊บ อัปเดตปั๊บ)

เปรียบเทียบ: เหมือนตาราง Microsoft Excel

สมมติเราตั้งสูตรช่องผลรวมไว้ เมื่อเราเปลี่ยนตัวเลขในช่อง A ค่าผลรวมในช่องยอดสุทธิ "**จะเปลี่ยนตามทันทีอัตโนมัติ**" โดยที่เราไม่ต้องสั่งคำนวณใหม่

Vue ก็ทำแบบนี้กับหน้าเว็บ! ข้อมูลหลังบ้านเปลี่ยนปุ๊บ หน้าจอฝั่งผู้ใช้จะอัปเดตให้เห็นทันที

```
import { ref } from 'vue'
const viewCount = ref(0)
viewCount.value++
```

// สร้างตัวแปรยอดเข้าชมเว็บ ที่เป็น Reactive
// เมื่อมีคนเข้าเว็บ เพิ่มค่าขึ้น 1 หน้าจอจะเปลี่ยนเลขทันที

คำสั่งพื้นฐานหน้าจอ (Vue Template Syntax)

1. การแสดงข้อมูล (Data Binding)

ใช้ปีกกาคู่ {{ }} เพื่อนำข้อมูลจากสมอง (Script) มาโชว์บนหน้าเว็บ

```
<h1>{{message}}</h1>
```

2. การกดปุ่มทำงาน (Events)

ใช้ @click หรือสัญลักษณ์ @ ตามด้วยเหตุการณ์ เพื่อสั่งงาน

```
<button> @click = "increase" เพิ่มจำนวน </button>
```

3. การวนลูป (Looping)

ใช้ v-for เพื่อสร้างรายการซ้ำๆ เช่น ตารางข้อมูลสินค้า

```
<li v-for="item in items"> {{ item.name }} </li>
```

4. การตั้งเงื่อนไข (Conditions)

ใช้ v-if และ v-else เพื่อซ่อน/แสดงข้อมูลบางอย่าง

```
<p v-if="isLogin">ยินดีต้อนรับเข้าสู่ระบบ</p>
<p v-else>กรุณาเข้าสู่ระบบก่อน</p>
```

โครงสร้างโปรเจกต์ (Project Structure)

เปรียบเทียบ: การเขียนโปรแกรมที่ดีเหมือน "การจัดระเบียบบ้าน"

เราไม่เอาเสื้อผ้าไปเก็บในห้องครัว และไม่เอาเตาแก๊สมาไว้ในห้องนอน

โฟลเดอร์ใน Vue ก็มีการแบ่งหน้าที่ห้องต่างๆ อย่างชัดเจน!

เจาะลึก ห้องต่างๆ ในบ้าน (ส่วนแสดงผล)

1. โฟลเดอร์ components/

(ชิ้นส่วนย่อยของบ้าน)

ใช้เก็บ UI ย่อยๆ ที่สร้างขึ้นมาเพื่อ "นำไปใช้ซ้ำ" ในหลายๆ ที่

- BaseButton.vue (ปุ่มกดมาตรฐาน)
- ProductCard.vue (การ์ดแสดงสินค้า)
- Navbar.vue (แถบเมนูด้านบน)
- UsersTable.vue (ตารางแสดงผลรายการ)

2. โฟลเดอร์ views/

(ห้องต่างๆ ของบ้าน)

ใช้เก็บ "หน้าจอแบบเต็มหน้า" ซึ่งหน้านี้เกิดจากการเอา components หลายๆ อันมาประกอบร่างรวมกัน

- LoginView.vue (หน้าล็อกอิน)
- DashboardView.vue (หน้ากระดานสรุปผล)
- ProductView.vue (หน้าจัดการสินค้า)
- UserManagement.vue (หน้าจัดการผู้ใช้งาน)

เจาะลึก ห้องต่างๆ ในบ้าน (ส่วนระบบงาน)

3. services/

(พนักงานรับส่งของ)

โฟลเดอร์นี้มีไว้เขียนโค้ดเพื่อ **เรียกขอข้อมูลจาก Backend** (API) แยกออกมาเพื่อให้จัดการง่าย ไม่ปนกับหน้าจอ

4. router/

(แผนที่นำทาง)

ใช้จัดการ **การเปลี่ยนหน้า (Routing)** กำหนดว่าถ้าผู้ใช้พิมพ์ URL นี้ จะพาไปเปิดห้อง (views) ห้องไหน

5. store/

(คลังเก็บของส่วนกลาง)

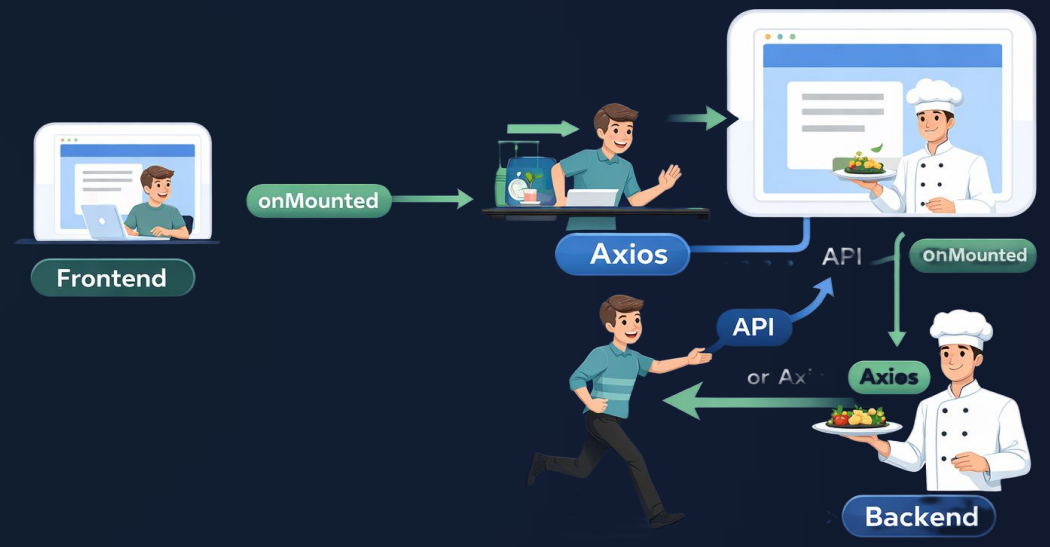
ใช้เก็บ **ข้อมูลส่วนกลางที่ต้องแชร์กัน (State Management)** เช่น ข้อมูลของคนที่ล็อกอินอยู่ (Pinia)

การเชื่อมต่อ Backend (API Integration)

กระบวนการขอข้อมูล (Axios & Lifecycle)

เปรียบเทียบ: เหมือนการไปทานอาหารที่ร้าน

- เรา (Frontend) นั่งที่โต๊ะ หน้าจอโหลดเสร็จ (เรียกว่าช่วง onMounted)
- เราจดออเดอร์ให้พนักงานเสิร์ฟ (เรียกใช้งาน Axios เพื่อยิง API)
- พนักงานวิ่งไปบอกพ่อครัว (Backend) ให้ทำอาหาร (ดึงข้อมูล)
- พ่อครัวทำเสร็จ พนักงานยกอาหารมาเสิร์ฟ หน้าจอเราอัปเดตข้อมูลขึ้นมาทันที!




```vue
import { onMounted } from 'vue'
onMounted(() => {
getUsers() })
```

// เมื่อลูกค้ามานั่งโต๊ะแล้ว (หน้าจอเปิดขึ้นมา)
// สั่งพนักงานไปเอาข้อมูลพนักงานทั้งหมดมาให้หน่อย

ตัวอย่างระบบ : ระบบจัดการสินค้า (Product CRUD)

ระบบประกอบด้วย 3 ฟีเจอร์หลัก

- **โชว์สินค้า (Show)**: ดึงข้อมูลจาก API มาวนลูปโชว์ในตารางด้วย v-for
- **เพิ่มสินค้า (Add)**: สร้างฟอร์มให้กรอกข้อมูล แล้วกดปุ่มบันทึก
- **ลบสินค้า (Delete)**: มีปุ่มถังขยะข้างๆ สินค้า พอกด @click ให้ข้อมูลหายไป

ทริค: อย่าลืมแยกหน้าจอ (views) ออกจากโค้ดยิง API (services)

ตัวอย่างโค้ดหน้าตาตารางสินค้า

```
<tr v-for="item in products" :key="item.id">
  <td>{{ item.name }}</td>
  <td>{{ item.price }} บาท</td>
  <td>
    <button @click="deleteProduct(item.id)">ลบ
</button>
  </td>
</tr>
```

กฎเหล็ก 5 ข้อ (Best Practice Frontend)

1. แยก Component เสมอ อะไรที่ใช้ซ้ำได้ หรือหน้าจอเริ่มยาวเกินไป ให้จับแยกเป็นชิ้นส่วน (Component) เล็กๆ เพื่อให้ง่ายต่อการแก้ไข

2. ใช้ TypeScript เสมอ แม้จะเขียนยากขึ้นนิดหน่อย แต่คุ้มค่ามหาศาล เพราะช่วยป้องกันบั๊ก (Bug) และช่วยให้คนในทีมพิมพ์โค้ดได้เร็วขึ้น (Auto Complete)

3. แยก API ออกจากหน้า UI (แยก Service) อย่าเขียนโค้ดเรียกขอข้อมูลไว้ในไฟล์หน้าจอ ให้แยกไปไว้ในโฟลเดอร์ services เพื่อความสะอาดและนำไปใช้ในหน้าอื่นต่อได้

4. ตั้งชื่อไฟล์และตัวแปรให้ชัดเจน (Naming) เช่น หน้าจอลงชื่อเข้าใช้ ควรตั้งชื่อ LoginView.vue ไม่ใช่ Page1.vue

5. เขียนโค้ดให้ Reusable คิดเผื่ออนาคตเสมอ ว่าฟังก์ชันนี้ หรือกล่องข้อความนี้ จะมีโอกาสถูกเอาไปใช้ซ้ำในหน้าอื่นๆ อีกหรือไม่

บทสรุป: Vue + TypeScript เหมาะกับงานแบบไหน?

Dashboard & Admin Panel

ระบบจัดการหลังบ้านที่ต้องการความรวดเร็วและดูข้อมูลสรุปจำนวนมาก

Enterprise Web / Inventory

ระบบระดับองค์กรที่ซับซ้อน เช่น จัดการคลังสินค้าที่มีข้อมูลเชื่อมโยงกัน

POS System (จุดขาย)

แอพพลิเคชั่นหน้าจอบน Tablet ที่ต้องตอบสนองรวดเร็วเวลาลูกค้าคิดเงิน

"โค้ดอ่านง่าย - เรียนรู้เร็ว - รักษาระยะยาวสบาย (Maintain ง่าย)"

Q&A / จบการอบรม

มีคำถามเพิ่มเติม หรืออยากเจาะลึกส่วนไหนเป็นพิเศษไหมครับ?